INFORME DE TESTING DEL SISTEMA VETERINARIO

1. Información General

Proyecto: Aplicación Veterinaria ITQ

Tecnología Backend:

- Java 17
- Servlets Jakarta
- Maven
- PostgreSQL
- Apache Tomcat 10

Herramientas de prueba:

- IntelliJ IDEA
- PostgreSQL
- Postman
- Apache Tomcat 10.1

2. Objetivo

Verificar el correcto funcionamiento de la API REST del Sistema Veterinario, comprobando la conectividad con la base de datos PostgreSQL, el despliegue en Apache Tomcat y las operaciones CRUD expuestas mediante los endpoints REST.

3. Entorno de Pruebas

Elemento	**Valor**
IDE	**Intellij Idea**
Servidor	**Apache Tomcat 10.1**
Base de datos	**PostgreSQL**
Lenguaje	**Java 24**
API	**REST**
Herramienta Testing	**Posman**

4. Resumen de Casos de Prueba

ID	PRUEBA	RESULTADO
CP-01	CONEXIÓN PostgreSQL	✅ APROBADO
CP-02	Despliegue Tomcat	✅ APROBADO
CP-03	GET Mascotas	✅ APROBADO

CP-04	UUID inválido	✅ APROBADO
CP-05	Registro inexistente	✅ APROBADO
CP-06	Crear Mascota	✅ APROBADO
CP-07	Crear Mascota sin nombre	❌ Defecto encontrado
CP-08	Fecha Futura	❌ Defecto encontrado
CP-09	Registro de Mascota con cliente inexistente	✅ APROBADO
CP-10	Actualización de mascota	✅ APROBADO
CP-11	Actualización de Mascota Inexistente	✅ APROBADO
CP-12	Actualizar Mascota sin Nombre	❌ Defecto encontrado
CP-13	Actualización de Mascota con fecha futura	❌ Defecto encontrado
CP-14	Eliminación de mascota existente	✅ APROBADO
CP-15	Verificación de eliminación física	✅ APROBADO
CP-16	Eliminación con identificador invalido	✅ APROBADO
CP-17	Eliminación de mascota Inexistente	✅ APROBADO

Desarrollo de Pruebas

CP-01 Conexión PostgreSQL

Objetivo

Verificar la comunicación entre el backend Java y la base de datos PostgreSQL.

Procedimiento

1. Configurar las credenciales en ConexionBD.java
2. Ejecutar PruebaConexion.java
3. Realizar consulta de validación

Resultado Esperado
Conexión exitosa a PostgreSQL

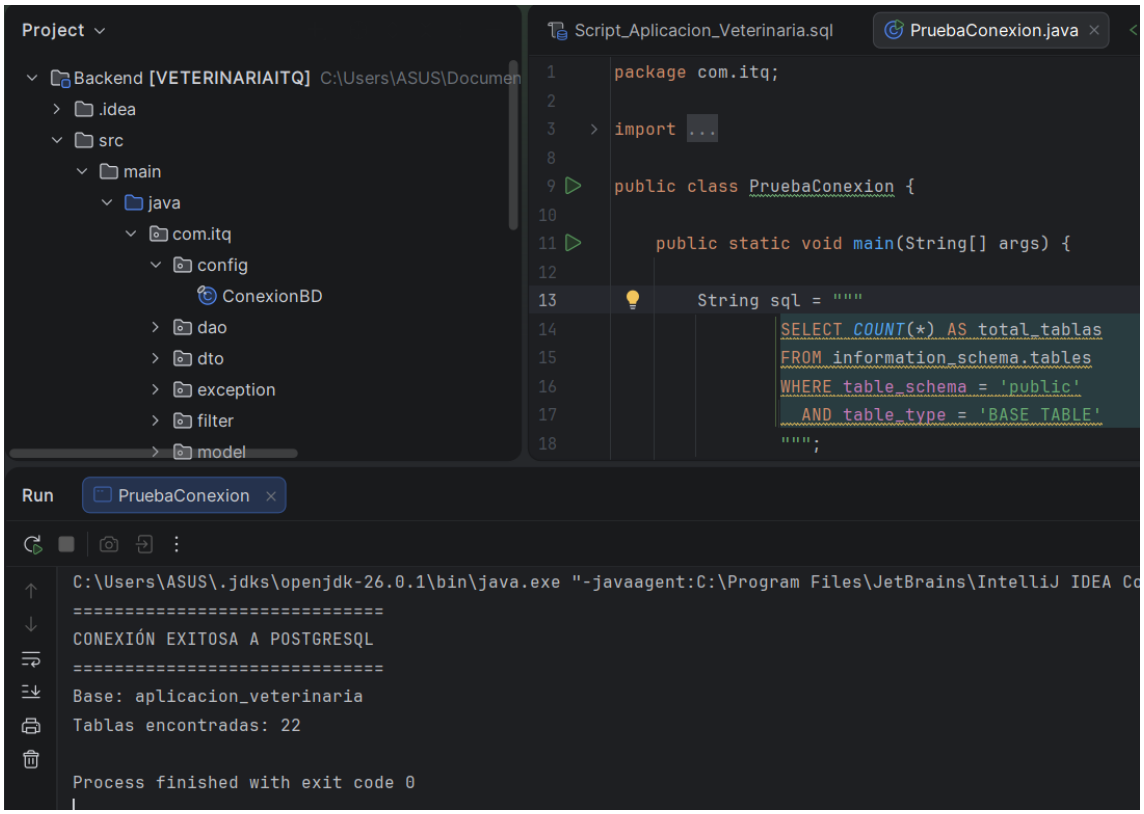
Resultado Obtenido

CONEXIÓN EXITOSA A POSTGRESQL
Base: aplicacion_veterinaria
Tablas encontradas: 22

Estado

✅ APROBADO

Evidencia



CP-02 Despliegue Tomcat

Objetivo

Verificar que la aplicación pueda desplegarse correctamente.

Procedimiento

1. Configurar Apache Tomcat 10.1
2. Ejecutar la aplicación
3. Verificar el inicio del servidor

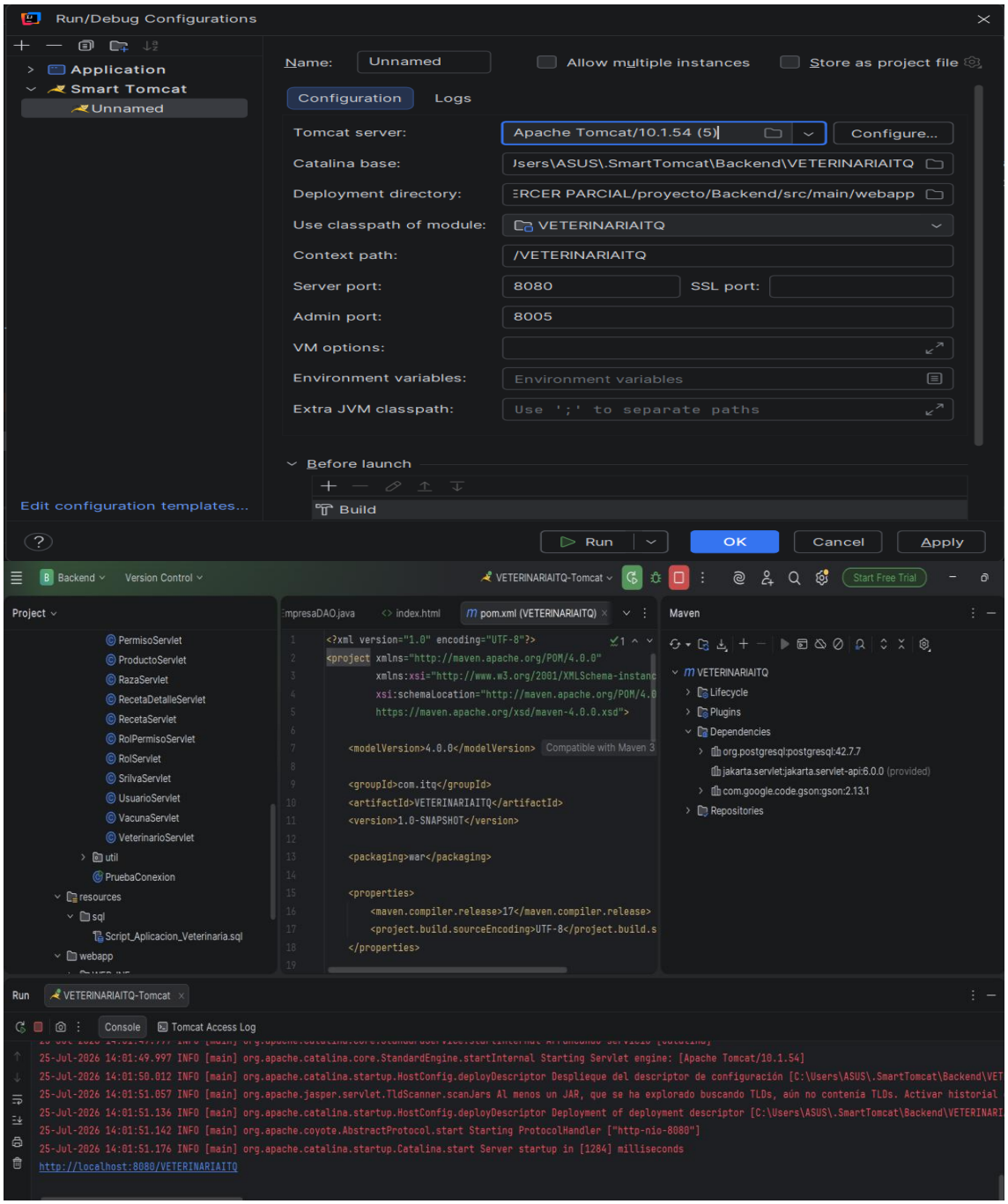
Resultado Esperado
Servidor iniciado correctamente

Resultado Obtenido
Server startup in 1284 milliseconds
http://localhost:8080/VETERINARIAITQ

Estado

✅ APROBADO

Evidencia



CP-03 Endpoint GET Mascotas

Objetivo

Verificar la obtención del listado de mascotas.

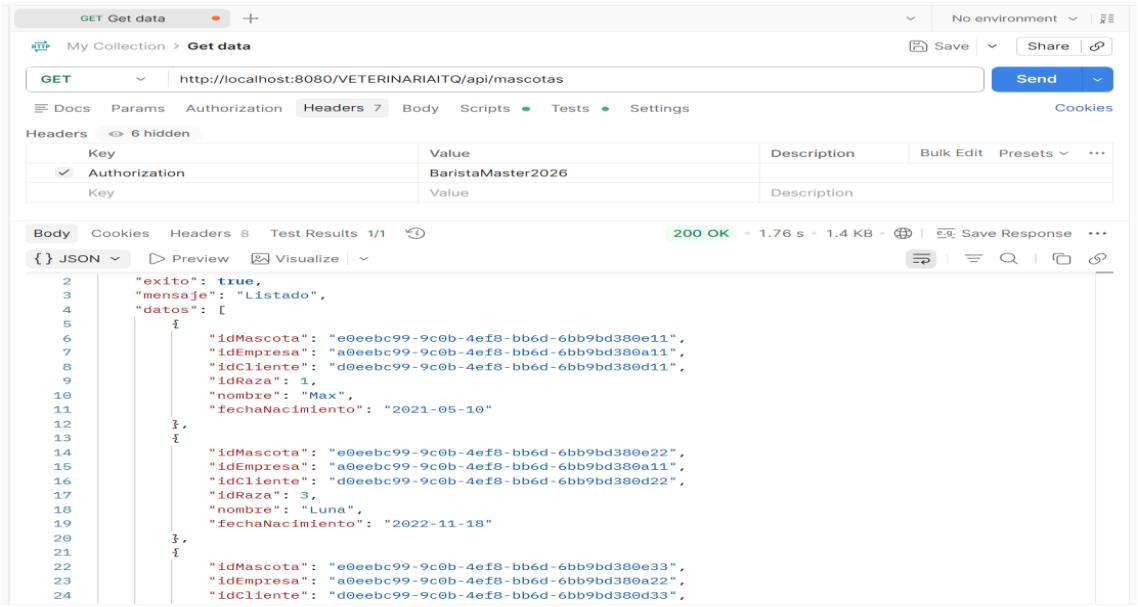
Endpoint
http://localhost:8080/VETERINARIAITQ/api/mascotas

Resultado Obtenido
{
"exito": true,
"mensaje": "Listado",
"datos": [...]
}

Estado

✅ APROBADO

Evidencia



CP-04 Consulta con identificador inválido

Objetivo

Verificar que el sistema valide correctamente los identificadores enviados por el cliente.

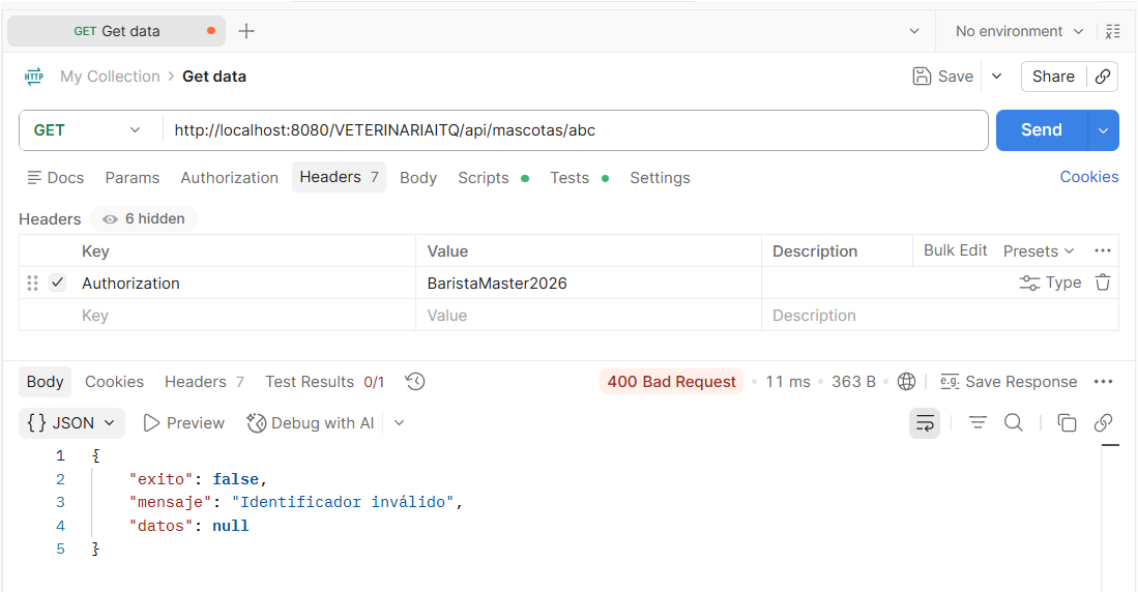
Endpoint
GET /api/mascotas/abc

Resultado obtenido
400 Bad Request

Estado

✅ APROBADO

Evidencia



CP-05 Consulta de Mascota Inexistente

Objetivo

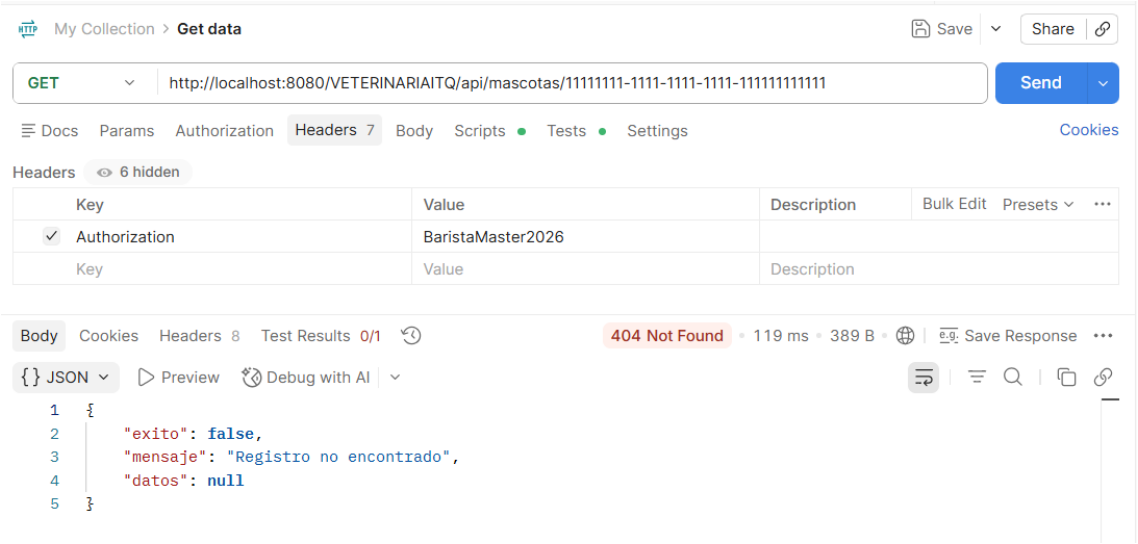
Verificar la respuesta del sistema cuando se consulta un registro inexistente.

Endpoint
GET /api/mascotas/11111111-1111-1111-1111-111111111111
Resultado obtenido
404 Not Found
{
"exito": false,
"mensaje": "Registro no encontrado"
}

Estado

✅ APROBADO

Evidencia



CP-06 Registro de Mascota

Objetivo

Verificar la creación de una mascota mediante el endpoint REST.

Endpoint

POST /api/mascotas

Resultado obtenido

201 Created

{

"exito": true,

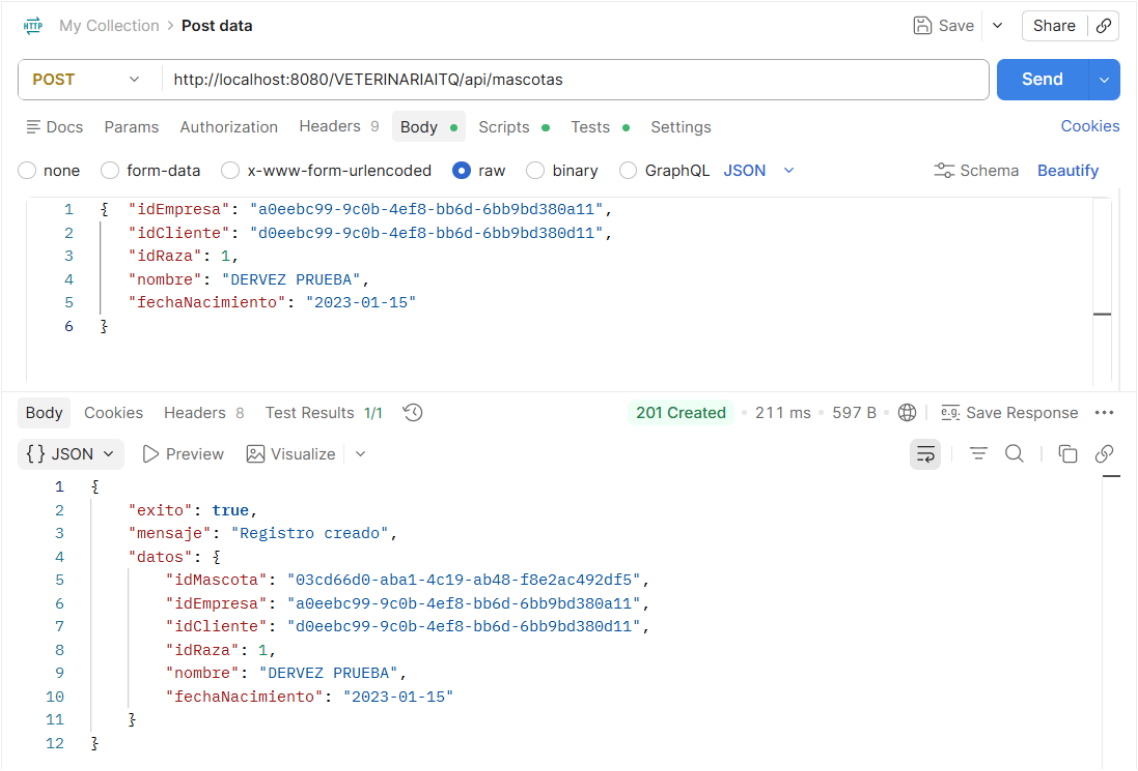
"mensaje": "Registro creado"

}

Estado

✅ APROBADO

Evidencia



CP- 09 Validación de Integridad Referencial (Cliente Inexistente)

Objetivo

Verificar que el sistema impida registrar una mascota asociada a un cliente no existente en la base de datos.

Endpoint

POST /api/mascotas

Datos de Entrada

{
"idEmpresa": "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11",
"idCliente": "11111111-1111-1111-1111-111111111111",
"idRaza": 1,
"nombre": "CLIENTE INVALIDO",
"fechaNacimiento": "2020-01-01"
}

Resultado Obtenido

Código HTTP:

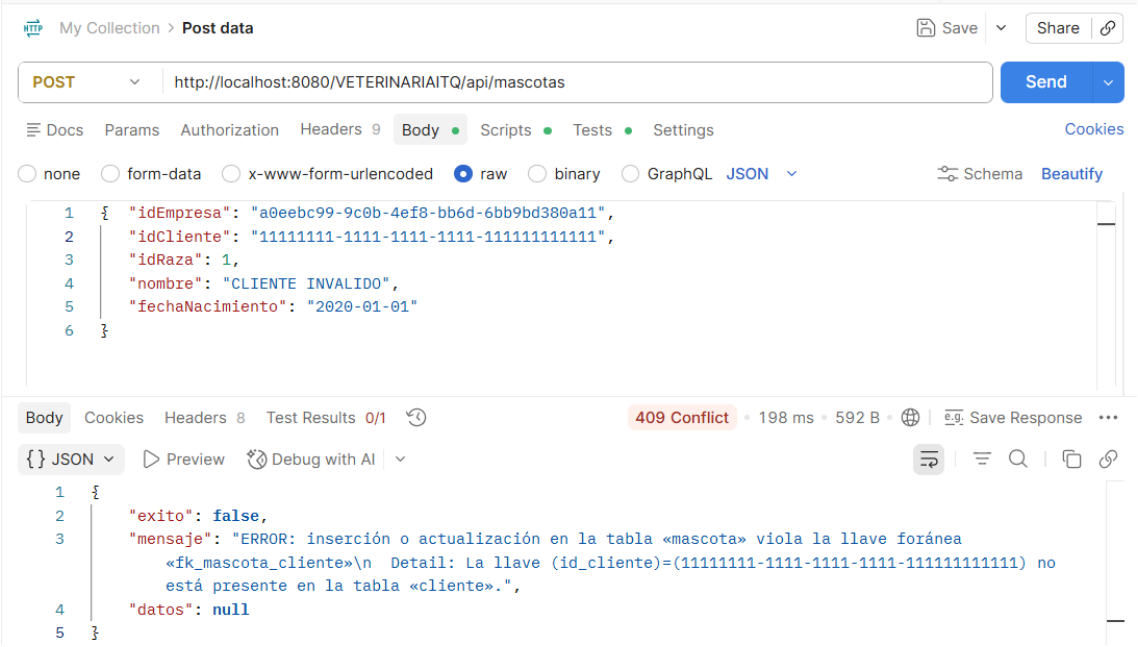
409 Conflict

{
"exito": false,
"mensaje": "ERROR: inserción o actualización en la tabla «mascota» viola la llave foránea «fk_mascota_cliente». Detalle: La llave (id_cliente)=(11111111-1111-1111-1111-111111111111) no está presente en la tabla «cliente».",
"datos": null
}

Estado

✅ APROBADO

Evidencia



CP-10 Actualización de Mascota

Objetivo

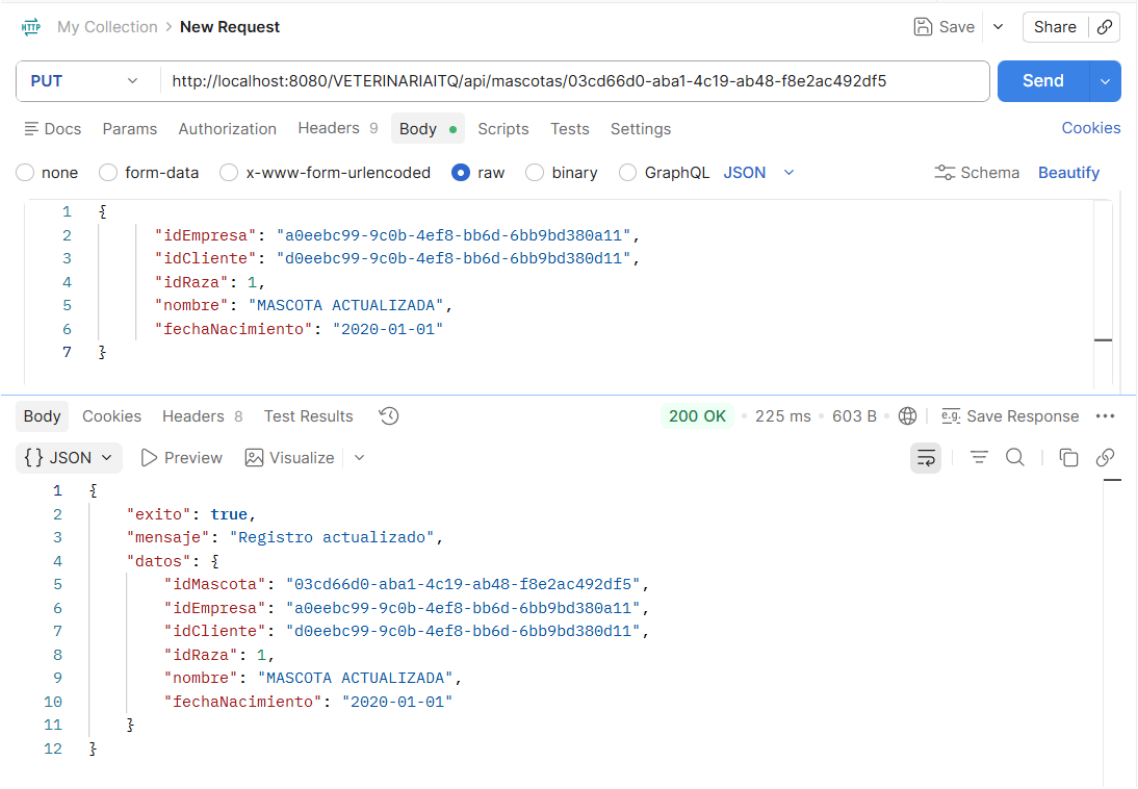
Verificar que el sistema permita modificar correctamente los datos de una mascota existente.

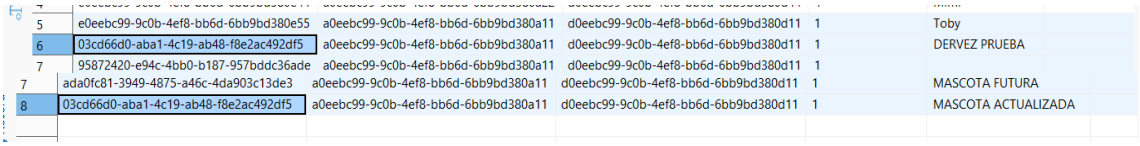
Endpoint

PUT /api/mascotas/{idMascota}

Datos enviados
{
"idEmpresa": "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11",
"idCliente": "d0eebc99-9c0b-4ef8-bb6d-6bb9bd380d11",
"idRaza": 1,

"nombre": "MASCOTA ACTUALIZADA",
"fechaNacimiento": "2020-01-01"
}
Resultado obtenido
200 OK
{
"exito": true,
"mensaje": "Registro actualizado"
}
y que el cambio sea reflejado en la base de datos.
Además, la consulta directa en PostgreSQL evidencia que el nombre cambió de:
DERVEZ PRUEBA a MASCOTA ACTUALIZADA





CP-11 Actualización de Mascota Inexistente

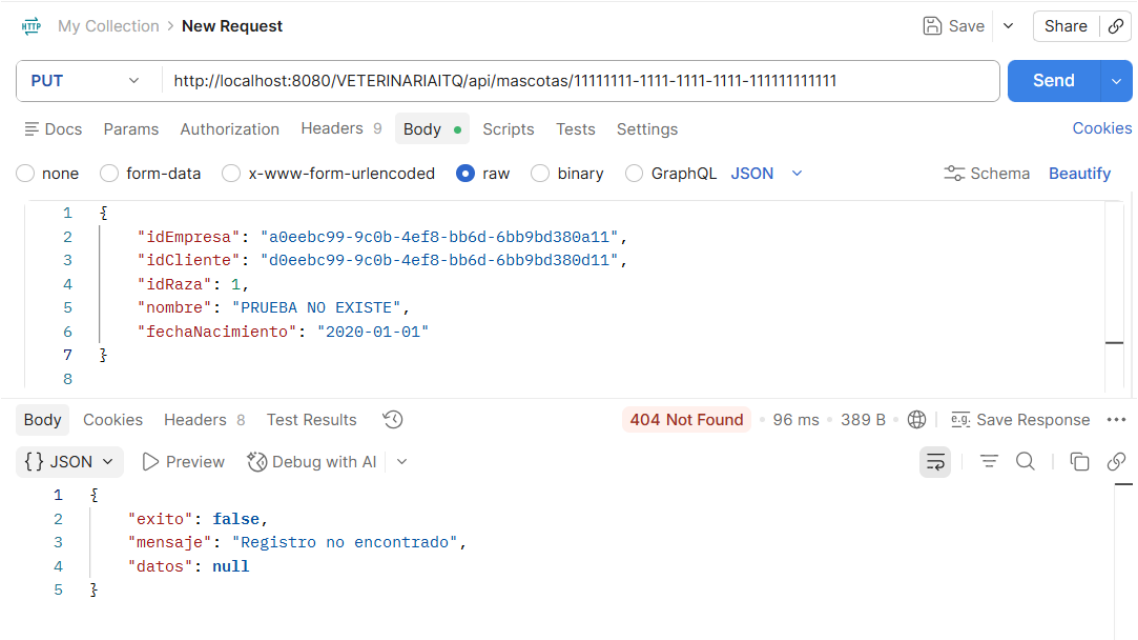
Objetivo

Verificar que el sistema impida la actualización de registros que no existen en la base de datos.

URL utilizada
http://localhost:8080/VETERINARIAITQ/api/mascotas/11111111-1111-1111-1111-111111111111
Resultado OBTENIDO
404 Not Found
{
"exito": false,
"mensaje": "Registro no encontrado"
}
El sistema verificó correctamente que el identificador no corresponde a una mascota existente.

Estado

✅ APROBADO



CP-14 Eliminación de Mascota Existente

Objetivo

Verificar que el sistema elimine correctamente una mascota registrada.

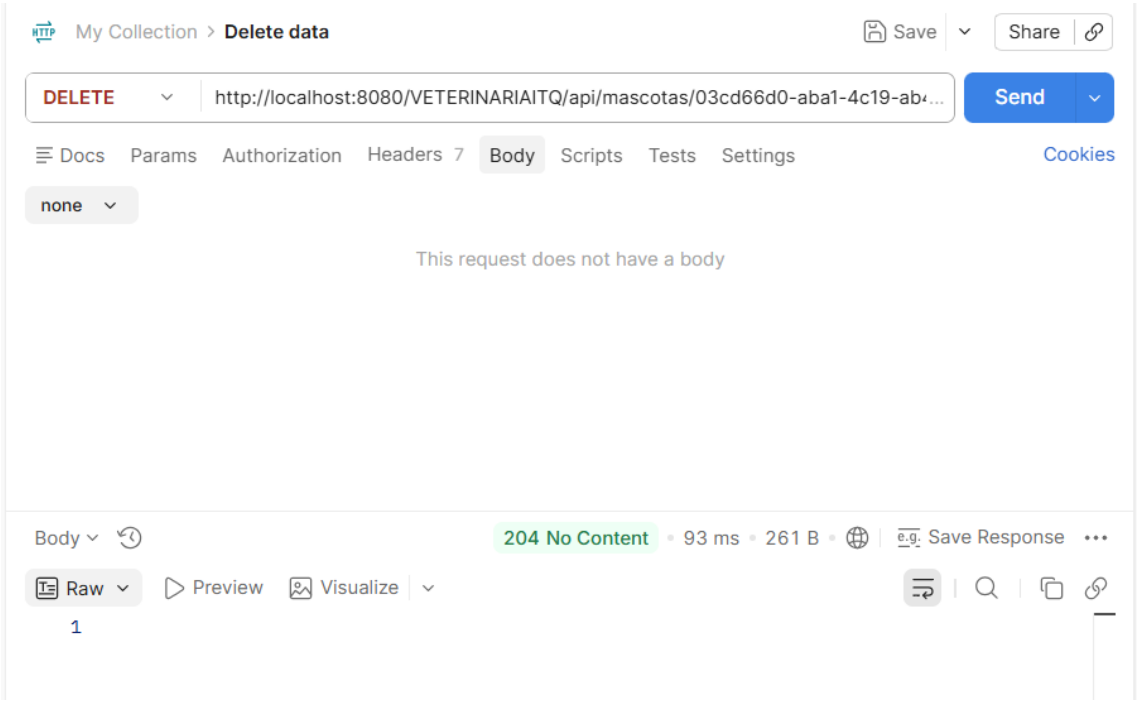
URL utilizada
http://localhost:8080/VETERINARIAITQ/api/mascotas/03cd66d0-aba1-4c19-ab48-f8e2ac492df5

Resultado obtenido
200 OK
204 No Content

Estado

✅ APROBADO

Evidencia



CP-15 Verificación de Eliminación Física

Objetivo

Verificar que una mascota eliminada no pueda volver a ser consultada mediante la API.

Endpoint

GET /api/mascotas/03cd66d0-aba1-4c19-ab48-f8e2ac492df5
Resultado obtenido
404 Not Found

{
"exito": false,
"mensaje": "Registro no encontrado",
"datos": null
}

Estado

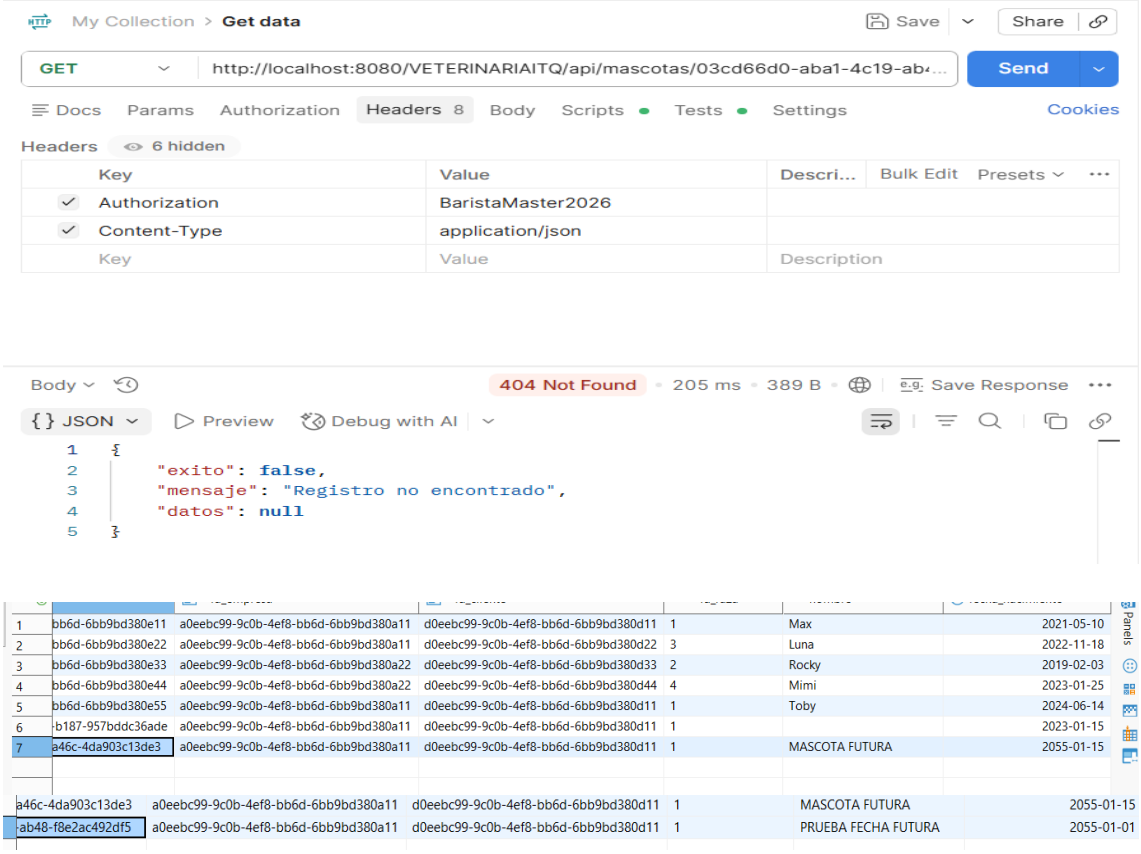
✅ APROBADO

Análisis

La prueba demuestra que:

- El DELETE ejecutado en CP-15 fue exitoso.
- El registro ya no existe en la base de datos.
- La API responde adecuadamente cuando se intenta consultar un registro eliminado.
- No se encontraron inconsistencias entre la API y PostgreSQL.

Evidencia



1	bb6d-6bb9bd380e11	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11	d0eebc99-9c0b-4ef8-bb6d-6bb9bd380d11	1	Max	2021-05-10
2	bb6d-6bb9bd380e22	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11	d0eebc99-9c0b-4ef8-bb6d-6bb9bd380d22	3	Luna	2022-11-18
3	bb6d-6bb9bd380e33	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a22	d0eebc99-9c0b-4ef8-bb6d-6bb9bd380d33	2	Rocky	2019-02-03
4	bb6d-6bb9bd380e44	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a22	d0eebc99-9c0b-4ef8-bb6d-6bb9bd380d44	4	Mimi	2023-01-25
5	bb6d-6bb9bd380e55	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11	d0eebc99-9c0b-4ef8-bb6d-6bb9bd380d11	1	Toby	2024-06-14
6	b187-957bddc36ade	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11	d0eebc99-9c0b-4ef8-bb6d-6bb9bd380d11	1		2023-01-15
7	a46c-4da903c13de3	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11	d0eebc99-9c0b-4ef8-bb6d-6bb9bd380d11	1	MASCOTA FUTURA	2055-01-15

a46c-4da903c13de3	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11	d0eebc99-9c0b-4ef8-bb6d-6bb9bd380d11	1	MASCOTA FUTURA	2055-01-15
ab48-f8e2ac492df5	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11	d0eebc99-9c0b-4ef8-bb6d-6bb9bd380d11	1	PRUEBA FECHA FUTURA	2055-01-01

CP-16 Eliminación con Identificador Inválido

Objetivo

Verificar que el sistema valide correctamente el formato del identificador antes de procesar una eliminación.

Endpoint

http://localhost:8080/VETERINARIAITQ/api/mascotas/11111111-1111-1111-1111-111111111111

Resultado obtenido

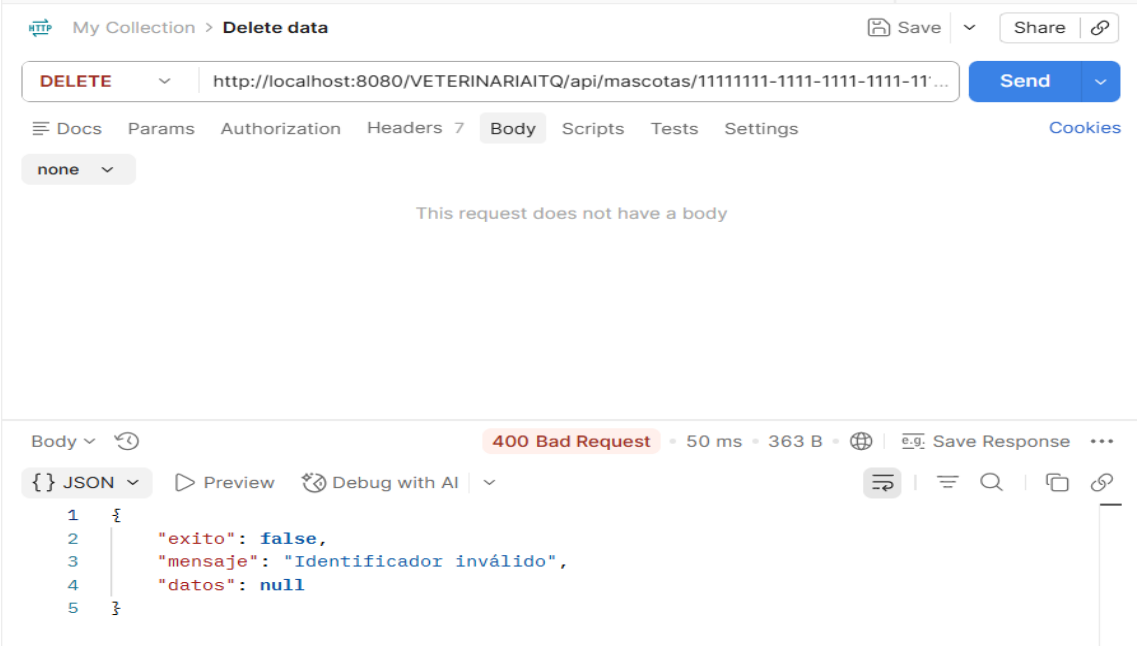
400 Bad Request

{
"exito": false,
"mensaje": "Identificador inválido"
}

Estado

✅ APROBADO

Evidencia



CP-17 Eliminación de Mascota Inexistente

Objetivo

Verificar que el sistema responda correctamente cuando se intenta eliminar una mascota que no existe en la base de datos.

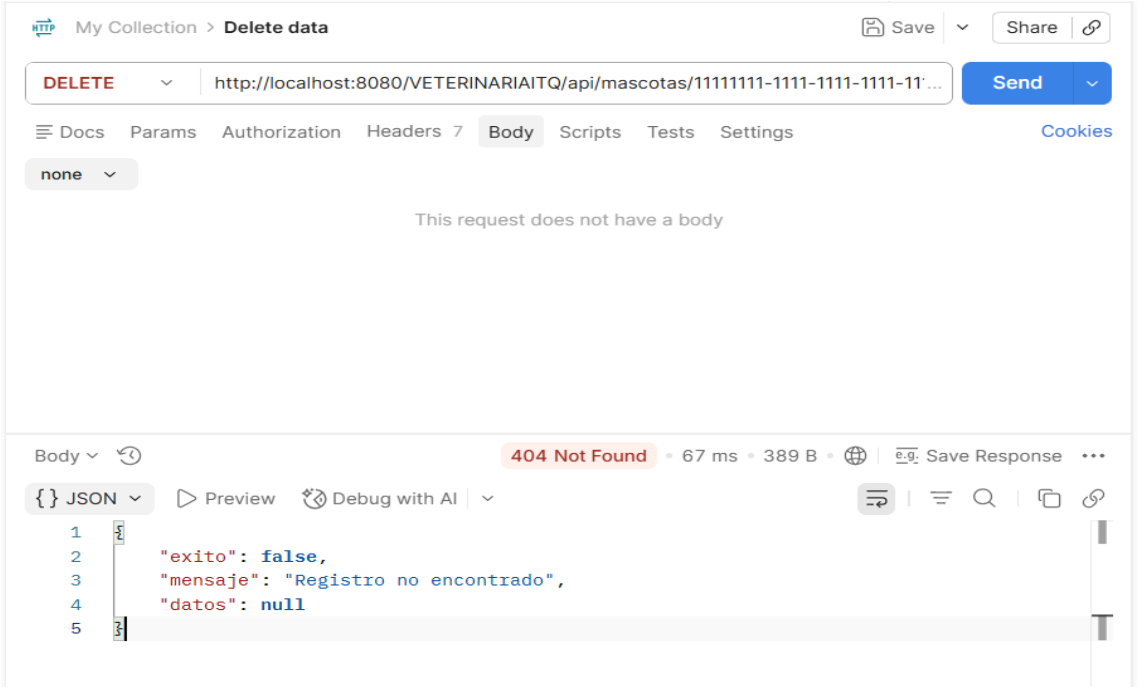
Endpoint
http://localhost:8080/VETERINARIAITQ/api/mascotas/11111111-1111-1111-1111-111111111111
Resultado obtenido
404 Not Found

{
"exito": false,
"mensaje": "Registro no encontrado",
"datos": null
}

Estado

✅ APROBADO

Evidencia



Hallazgos o Defectos

DEF-01

Módulo

Mascotas

Tipo

Validación de Datos

Descripción

El sistema permite registrar mascotas con el campo nombre vacío.

Pasos para reproducir

Realizar solicitud POST a:

/api/mascotas

Enviar:


```
{
  "idEmpresa": "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11",
  "idCliente": "d0eebc99-9c0b-4ef8-bb6d-6bb9bd380d11",
  "idRaza": 1,
  "nombre": "",
  "fechaNacimiento": "2023-01-15"
}
```


Resultado obtenido

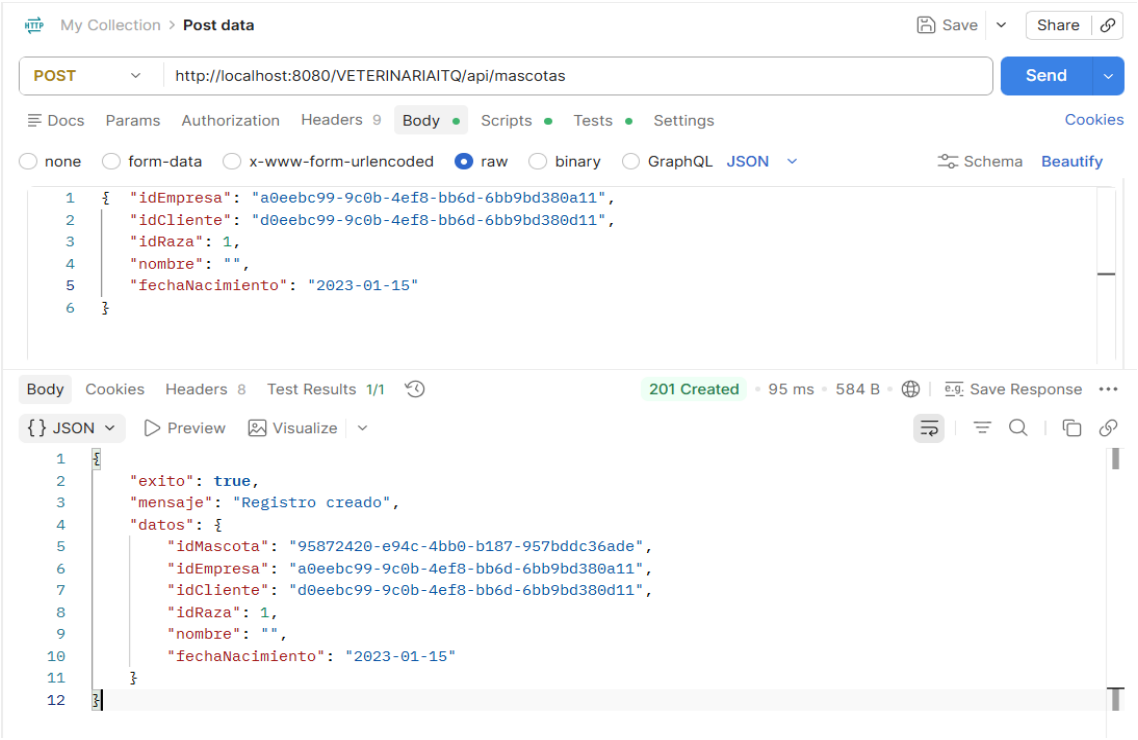
El sistema registra la mascota exitosamente.

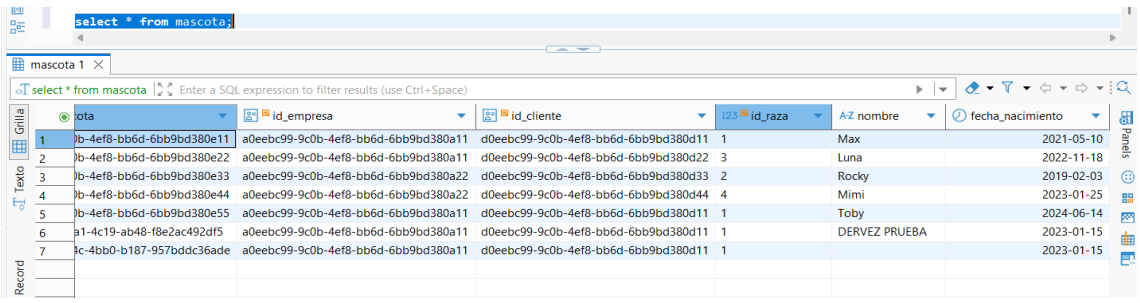
Severidad

Media

Recomendación

Validar que el campo nombre no sea nulo ni vacío antes de realizar la inserción en la base de datos.





DEF-02: Permite fechas de nacimiento futuras

Módulo

Mascotas

Severidad

Media

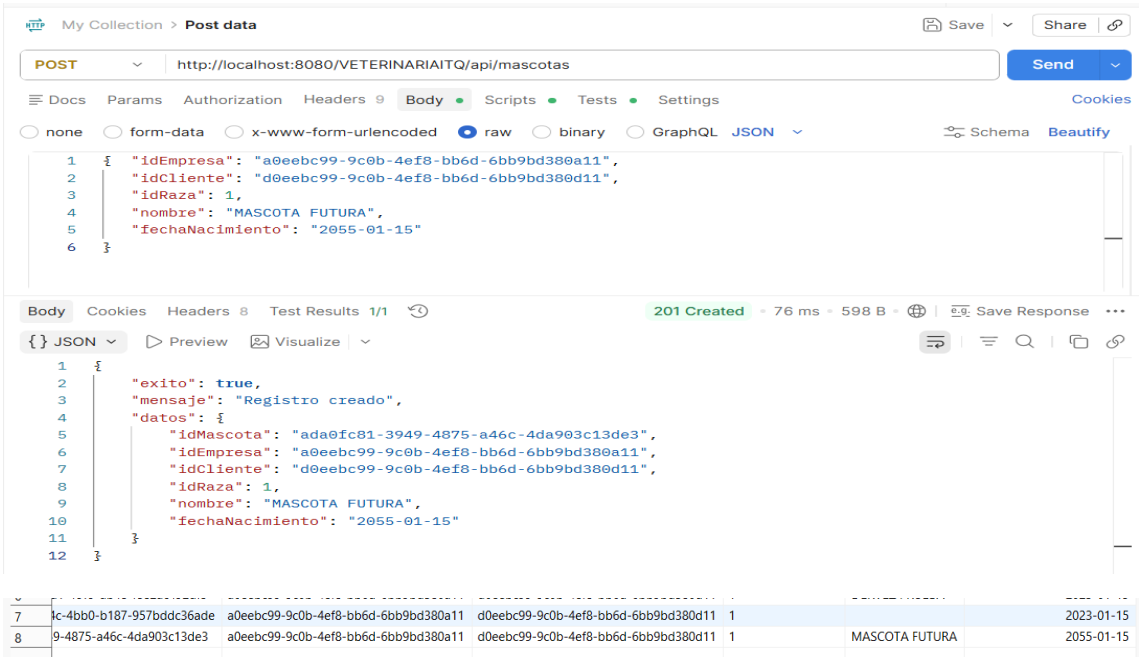
Descripción

La aplicación permite registrar mascotas con fechas futuras.

Resultado esperado
400 Bad Request
{
"mensaje":"La fecha de nacimiento no puede ser futura"
}
Resultado obtenido
201 Created

Recomendación

Implementar validaciones en el Service o Servlet:



DEF-03

Título

Permite actualizar mascotas con nombre vacío.

Módulo

Mascotas

Severidad

Media

Impacto

Permite que registros válidos se conviertan en registros inconsistentes después de una actualización.

Datos enviados

{
"idEmpresa": "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11",
"idCliente": "d0eebc99-9c0b-4ef8-bb6d-6bb9bd380d11",
"idRaza": 1,
"nombre": "",
"fechaNacimiento": "2020-01-01"
}

Resultado esperado

{
"exito": false,
"mensaje": "El nombre es obligatorio"
}

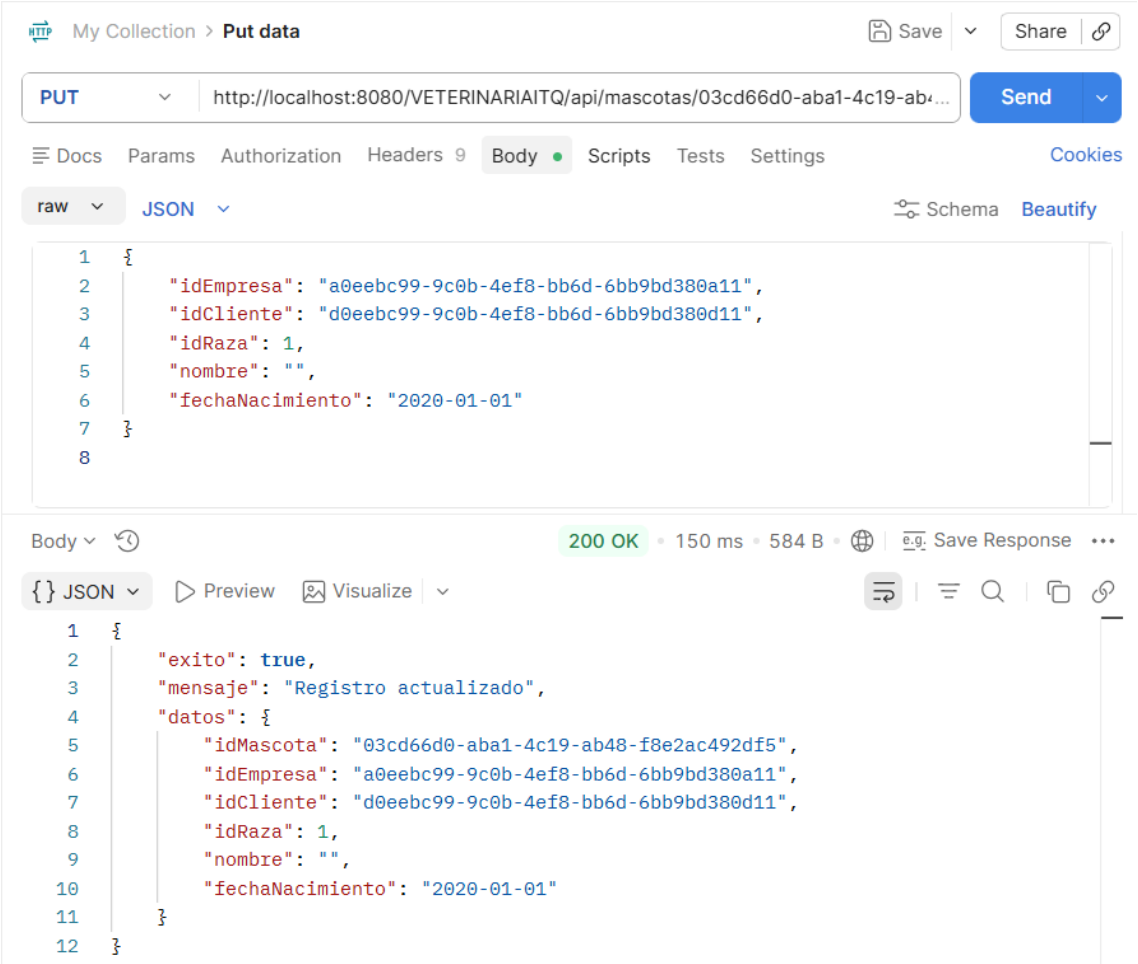
Resultado obtenido

{
"exito": true,
"mensaje": "Registro actualizado"
}

Recomendación

Validar:

nombre != null && !nombre.trim().isEmpty()
antes de ejecutar la actualización.



DEF-04

Título

Permite actualizar mascotas con fecha de nacimiento futura.

Módulo

Mascotas

Severidad

Media

Impacto

Permite almacenar información inconsistente en la historia clínica y reportes del sistema.

Datos enviados

{
"idEmpresa": "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11",
"idCliente": "d0eebc99-9c0b-4ef8-bb6d-6bb9bd380d11",
"idRaza": 1,
"nombre": "PRUEBA FECHA FUTURA",
"fechaNacimiento": "2055-01-01"
}

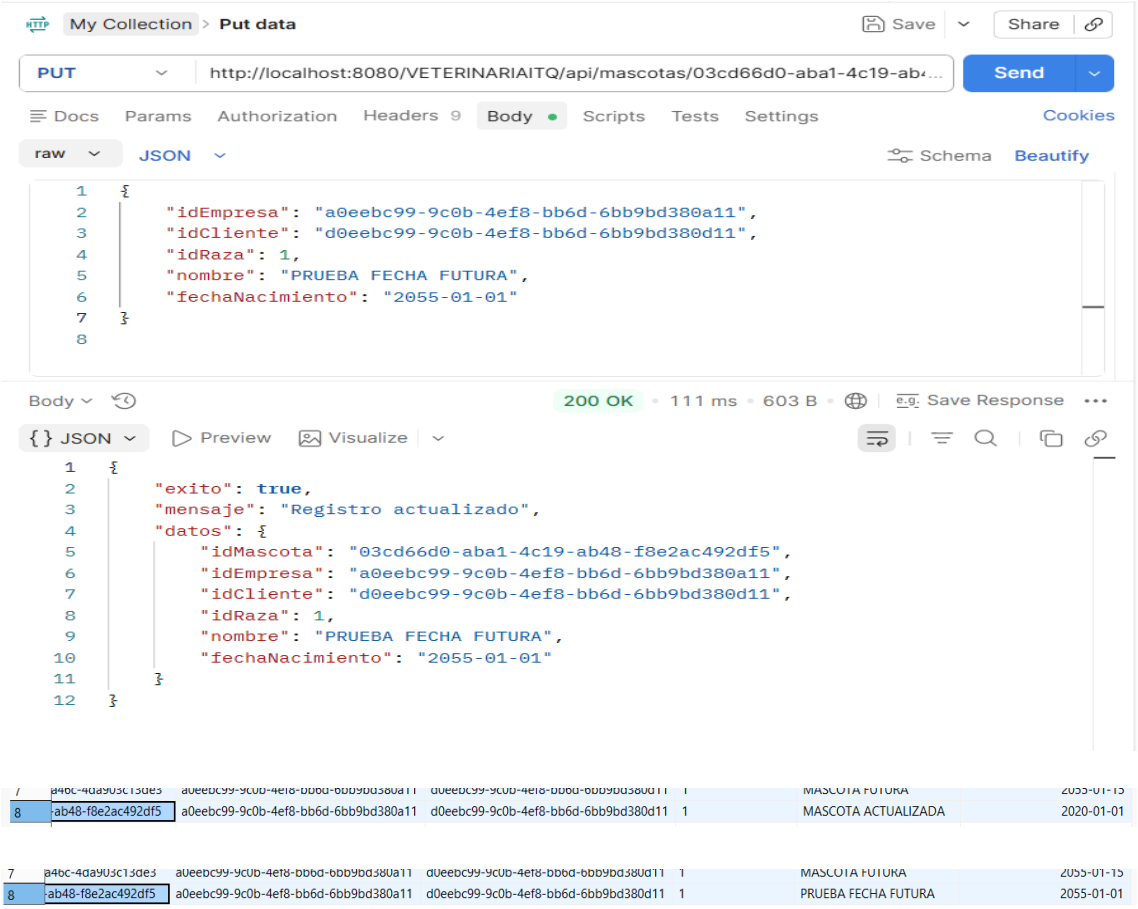
Resultado obtenido

{
"exito": true,
"mensaje": "Registro actualizado"
}

Recomendación

Implementar una validación:

if (fechaNacimiento.isAfter(LocalDate.now()))
antes de actualizar el registro.



8	ab48-f8e2ac492df5	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11	d0eebc99-9c0b-4ef8-bb6d-6bb9bd380d11	1	MASCOTA ACTUALIZADA	2020-01-01

7				1	MASCOTA FUTURA	2055-01-15
8	ab48-f8e2ac492df5	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11	d0eebc99-9c0b-4ef8-bb6d-6bb9bd380d11	1	PRUEBA FECHA FUTURA	2055-01-01

INFORME DE RESULTADOS DE TESTING

Sistema de Gestión Veterinaria ITQ

Se ejecutó un conjunto de pruebas funcionales sobre el módulo **Mascotas**, considerando que este módulo es uno de los componentes principales del sistema y mantiene relaciones directas con otras entidades como Cliente, Empresa y Raza.

Las pruebas realizadas permitieron validar la conectividad con la base de datos PostgreSQL, el funcionamiento de la API REST desplegada en Apache Tomcat, la correcta implementación de las operaciones CRUD (Crear, Consultar, Actualizar y Eliminar) y los mecanismos de control de integridad referencial.

Como resultado general, el sistema demostró un comportamiento estable en la mayoría de operaciones; sin embargo, se identificaron deficiencias relacionadas con la validación de datos de entrada que podrían afectar la calidad e integridad de la información almacenada.

Resultados Generales

Pruebas Ejecutadas

17

Pruebas Aprobadas

13

Defectos Detectados

4

Pruebas Validadas Correctamente

Durante la ejecución del proceso de testing se verificó de manera satisfactoria:

Conectividad

- Conexión exitosa entre la aplicación Java y PostgreSQL.
- Identificación correcta de las tablas de la base de datos.
- Comunicación adecuada mediante JDBC.

Despliegue

- Ejecución correcta de la aplicación en Apache Tomcat 10.
- Disponibilidad de los servicios REST publicados.

Operaciones de Consulta

- Obtención correcta del listado de mascotas.
- Consulta de registros individuales.
- Manejo adecuado de identificadores inválidos.
- Manejo adecuado de registros inexistentes.

Operaciones de Creación

- Registro correcto de nuevas mascotas.
- Generación automática de identificadores UUID.
- Persistencia correcta de datos en PostgreSQL.

Integridad Referencial

- Protección efectiva de claves foráneas.
- Impedimento de registros asociados a clientes inexistentes.
- Respuesta adecuada mediante códigos HTTP de error.

Operaciones de Actualización

- Modificación correcta de registros existentes.
- Validación de registros inexistentes.
- Persistencia correcta de cambios en la base de datos.

Operaciones de Eliminación

- Eliminación física de registros.
- Verificación posterior de inexistencia del registro.
- Manejo correcto de intentos de eliminación sobre registros inexistentes.

Defectos Encontrados

DEF-01

Registro de mascotas con nombre vacío
Descripción
El sistema permite crear una mascota sin especificar nombre.
Resultado obtenido
La API respondió:
201 Created
permitiendo almacenar el registro
Impacto
Se generan registros incompletos y de difícil identificación.

DEF-02

Registro de mascotas con fecha de nacimiento futura
Descripción
El sistema permite registrar mascotas utilizando fechas posteriores a la fecha actual.
Resultado obtenido
La API respondió:
201 Created
guardando la información.
Impacto
Se almacena información inconsistente que puede afectar reportes, historiales clínicos y procesos de atención.

DEF-03

Actualización de mascotas con nombre vacío
Descripción
La aplicación permite modificar mascotas existentes dejando el campo nombre vacío.
Resultado obtenido
La API respondió:
200 OK
actualizando el registro exitosamente.
Impacto
Registros válidos pueden convertirse en registros incompletos tras una actualización.

DEF-04

Actualización de mascotas con fecha futura
Descripción
La aplicación permite modificar una mascota estableciendo una fecha de nacimiento futura.
Resultado obtenido
La API respondió:
200 OK
y almacenó la nueva fecha.
Impacto
Produce inconsistencias en la información clínica del sistema.

Análisis General

Los resultados obtenidos evidencian que la arquitectura del sistema se encuentra correctamente implementada respecto a:

- Comunicación cliente-servidor.
- Operaciones CRUD.
- Manejo de excepciones.
- Integridad referencial.
- Conexión con PostgreSQL.
- Respuestas HTTP.

No obstante, se identificó una ausencia de validaciones de negocio sobre datos de entrada, principalmente en los procesos de creación y actualización de mascotas.

Las deficiencias encontradas no comprometen la estabilidad de la aplicación, pero sí afectan directamente la calidad y consistencia de los datos almacenados.

Recomendaciones

Se recomienda implementar validaciones tanto en la capa de servicios como en la base de datos para garantizar la integridad de la información.

Validaciones sugeridas

- No permitir nombres vacíos.
- No permitir valores nulos en campos obligatorios.
- Validar longitud mínima y máxima de textos.
- Impedir fechas de nacimiento futuras.
- Validar consistencia de los datos antes de realizar inserciones o actualizaciones.
- Mantener mensajes de error amigables para el usuario final.

Conclusión

El sistema veterinario presentó un comportamiento funcional satisfactorio en las operaciones principales de consulta, creación, actualización, eliminación e integridad referencial. Las pruebas ejecutadas confirmaron el correcto funcionamiento de la infraestructura tecnológica basada en Java, Jakarta Servlet, PostgreSQL y Apache Tomcat.

Sin embargo, se identificaron cuatro defectos relacionados con la validación de datos, los cuales deben ser corregidos antes de una liberación a producción con el fin de garantizar la calidad e integridad de la información gestionada por el sistema. En términos generales, el sistema puede considerarse estable desde el punto de vista funcional, requiriendo únicamente mejoras en las reglas de validación de negocio.